

Exception handling statements

PL/pgSQL can also handle exceptions. The `BEGIN . . . END` block of a function allows the `EXCEPTION` option, which works as a catch for exceptions. For example, if we write a function to divide two numbers, we could have a problem with a division by 0:

```
forumdb=> CREATE OR REPLACE FUNCTION my_first_except (x real, y real )
returns real as
$$
DECLARE
    ret real;
BEGIN
    ret := x / y;
    return ret;
END;
$$
language 'plpgsql';
CREATE FUNCTION
```

This function works well if $y \neq 0$, as we can see here:

```
forumdb=> \x
Expanded display is off.
forumdb=> select my_first_except(4,2);
 my_first_except
-----
                2
(1 row)
```

However, if y assumes a 0 value, we have a problem:

```
forumdb=> select my_first_except(4,0);
ERROR:  division by zero
CONTEXT:  PL/pgSQL function my_first_except(real,real) line 5 at
assignment
```

To solve this problem, we have to handle the exception. To do this, we have to rewrite our function in the following way:

```
forumdb=> CREATE OR REPLACE FUNCTION my_second_except (x real, y real )
returns real as
```

```

$$
DECLARE
    ret real;
BEGIN
    ret := x / y;
    return ret;
EXCEPTION
    WHEN division_by_zero THEN
        RAISE INFO 'DIVISION BY ZERO';
        RAISE INFO 'Error % %', SQLSTATE, SQLERRM;
        RETURN 0;
END;
$$
language 'plpgsql' ;
CREATE FUNCTION

```

The SQLSTATE and SQLERRM variables contain the status and message associated with the generated error. Now, if we execute the second function, we no longer get an error from PostgreSQL:

```

forumdb=> select my_second_except(4,0);
INFO:  DIVISION BY ZERO
INFO:  Error 22012 division by zero
 my_second_except
-----
                0
(1 row)

```

The list of errors that PostgreSQL can manage is available at <https://www.postgresql.org/docs/current/errcodes-appendix.html>.

Security definer

This option allows the user to invoke a function as if they were its owner. It can be useful in all cases where we want to display data to which the average user does not have access.

For example, in PostgreSQL, there is a system view called `pg_stat_activity`, which allows us to view what PostgreSQL is currently doing.

As user forum, let's execute this statement:

```

postgres@learn_postgresql:~$ psql -U forum forumdb

```

```

forumdb=>

forumdb=> select pid,query from pg_stat_activity ;
 pid |          query
-----+-----
  74 | <insufficient privilege>
  75 | <insufficient privilege>
 217 | select pid,query from pg_stat_activity ;
[...]
```

As we can see above, there are some <insufficient privilege> results. Here are the steps to solve this problem:

- Let's connect to the database as user postgres:

```

postgres@learn_postgresql:~$ psql forumdb
forumdb=#
```

- Now let's execute the function `my_stat_activity()` written here:

```

forumdb=# create function forum.my_stat_activity()
returns table (pid integer,query text)
as $$
    select pid, query  from pg_stat_activity;
$$ language 'sql'
security definer;
```

- Let's give the execute permission to the forum user on the function `my_stat_activity`. We will see this feature in *Chapter 10, Granting and Revoking Permissions*:

```

forumdb=# grant execute on function forum.my_stat_activity TO forum;
```

- Let's connect again to the database as user forum:

```

postgres@learn_postgresql:~$ psql -U forum forumdb
forumdb=>
```

- Now let's execute the query written below:

```

forumdb=> select * from my_stat_activity();
 pid |          query
-----+-----
```

```
74 |  
75 |  
271 | select * from my_stat_activity();  
[..]
```

We no longer have the problem we had before. This is because the `security definer` allows the `forum.my_stat_activity()` function to be executed with the permissions of the user who created it, and in this case, the user who created it is the `postgres` user.

Summary

In this chapter, we introduced the world of server-side programming. The topic is so vast that there are specific books dedicated just to it. We have tried to give you a better understanding of the main concepts of server-side programming. We talked about the main data types managed by PostgreSQL, then we saw how it is possible to create new ones using composite data types. We also mentioned SQL functions and polymorphic functions, and finally, we provided some information about the PL/pgSQL language.

In the next chapter, we will use these concepts to introduce event management in PostgreSQL. We will talk about event management through the use of triggers and the functions associated with them.

Verify your knowledge

- Is it possible to extend features and data types in postgresql?

Yes it is, we can extend PostgreSQL in terms of data types and in terms of functions.

See the *The concept of extensibility* section for more details.

- Does PostgreSQL support only relational databases?

No, PostgreSQL supports NoSQL databases too.

See the *The NoSql data type* section for more details.

- Does PostgreSQL support SQL functions?

Yes it does, we can write any kind of SQL function.

See the *SQL functions* section for more details.

- Does PostgreSQL have a default built-in procedural language?

Yes PostgreSQL has a default built-in procedural language called PL/pgSQL.

See the *PL/pgSQL functions* section for more details.

- As a user without administrative privileges, can we read a table that requires administrative permissions in order to be read?

Yes we can; as an administrator user let's create a function that reads the table, let's define the function using the security definer clause, and let's give the execution permissions of the function to the non-administrator user.

See the *Security definer* section for more details.

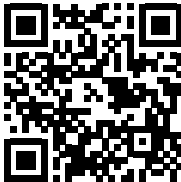
References

- PostgreSQL – data types official documentation: <https://www.postgresql.org/docs/current/datatype.html>
- PostgreSQL – SQL functions official documentation: <https://www.postgresql.org/docs/current/xfunc-sql.html>
- PostgreSQL – PL/pgSQL official documentation: <https://www.postgresql.org/docs/current/plpgsql.html>
- PostgreSQL 11 Server Side Programming Quick Start Guide: <https://subscription.packtpub.com/book/data/9781789342222/1>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



8

Triggers and Rules

In the previous chapter, we talked about server-side programming. In this chapter, we will use the concepts introduced in the previous chapter to manage the programming of events in PostgreSQL. The first thing we need to address is what an event in PostgreSQL actually is. In PostgreSQL, possible events are given by the `SELECT/INSERT/UPDATE`, and `DELETE` statements. There are also events related to **data definition language (DDL)** operations; however we will talk about those events in *Chapter 17, Event Triggers*.

In PostgreSQL, there are two ways to handle events:

- Rules
- Triggers

In this chapter, we will explore both of these ways and address when it is more appropriate to use one rather than the other. As a starting point, we can generally say that rules are usually simple event handlers, while triggers are more complex event handlers. Triggers and rules are often used to update accumulators and to modify or delete records that belong to different tables than the one in which we modify records. They are very powerful tools that allow us to perform operations in tables other than the one in which we modify the data. Triggers and rules will also be used in the next chapter when we talk about partitioning. This is because, in PostgreSQL, there is still a partitioning model based on triggers and rules.

In this chapter, we will talk about the following:

- Exploring rules in PostgreSQL
- Managing triggers in PostgreSQL
- Event triggers

Technical requirements

Before starting, remember to start the Docker container named `chapter_08`, as shown below:

```
$ bash run-pg-docker.sh chapter_08
postgres@learn_postgresql:~$ psql -U forum forumdb
```

Exploring rules in PostgreSQL

As mentioned earlier, rules are simple event handlers. At the user level, it is possible to manage all the events that perform write operations, which are as follows:

- INSERT
- DELETE
- UPDATE

The fundamental concept behind rules is to modify the flow of an event. If we are given an event, what we can do when certain conditions occur is as follows:

- Do nothing and then undo the action of that event.
- Trigger another event instead of the default one.
- Trigger another event in conjunction with the default.

So, given a write operation, for example, an INSERT operation, we can perform one of these three actions:

- Cancel the operation.
- Perform another operation instead of the INSERT.
- Execute the INSERT and, in the same transaction, perform another operation.

Understanding the OLD and NEW variables

Before we start working with rules and then with triggers, we need to understand the concept of the OLD and NEW variables.

The OLD and NEW variables represent the state of the row in the table before or after the event. OLD and NEW values are cursors that represent the whole record. To better understand this, consider an UPDATE operation; in this case, the OLD variable contains the value of the record already present in the table, while the NEW variable contains the value that the record of the table will have after the UPDATE operation.

For example, we can consider the tags table with the following records:

```
forumdb=> select * from tags;
pk |      tag      | parent
----+-----+-----
 1 | Operating Systems |
 2 | Linux          |      1
 3 | Ubuntu         |      2
[...]
```

Suppose we want to modify the tag with pk=3, from Ubuntu to Fedora, with this UPDATE operation:

```
forumdb=> update tags set tag='Fedora' where pk=3;
UPDATE 1
```

The OLD variable will have these values:

pk	tag	parent
3	Ubuntu	1

The NEW variable will have these values:

pk	tag	parent
3	Fedora	1

It is quite logical that, for certain operations, both the OLD variable and the NEW variable may exist, but for other operations, only one of them may exist. Here, we can see this expressed in more detail:

Operation/Variable	NEW	OLD
INSERT	present	absent
DELETE	absent	present
UPDATE	present	present

Now that everything is clearer, we can start working with rules.

Rules on INSERT

Let's start by introducing the rules syntax:

```
CREATE [ OR REPLACE ] RULE name AS ON event
    TO table [ WHERE condition ]
```

```
DO [ ALSO | INSTEAD ] { NOTHING | command | ( command ; command ... )  
}
```

As we can see, the rule definition is extremely simple. There are three options that we have when we decide to use a rule:

- The ALSO option
- The INSTEAD option
- The INSTEAD NOTHING option

The ALSO option

Suppose that, from the tags table, we want to copy all records with the field tag value starting with the letter a in the a_tag table:

1. First of all, let's create a new table called O_tags:

```
forumbd=> create table O_tags (  
    pk integer not null primary key,  
    tag text,  
    parent integer);  
CREATE TABLE
```

2. Then let's create the new rule as follows:

```
forumbd=> create or replace rule r_tags1  
as on INSERT to tags  
where NEW.tag ilike 'O%' DO ALSO  
insert into O_tags(pk,tag,parent)values (NEW.pk,NEW.tag,NEW.  
parent);  
CREATE RULE
```

In the rule we have just defined, we simply told PostgreSQL that every time a record is inserted with a tag value that starts with the letter “O,” as well as being inserted into the tags table, it must also be inserted into the O_tags table.

3. Now we perform the following query:

```
forumbd=> insert into tags (tag) values ('OpenBSD');  
INSERT 0 1
```

4. Then we check the records in the tags table and the O_tags records. We will find, in the tags table, the following:

```

forumbd=> select * from tags;
pk |      tag      | parent
----+-----+-----
  1 | Operating Systems |
  2 | Linux          |      1
  3 | Ubuntu         |      2
  4 | OpenBSD        |
(4 rows)

```

In the O_tags table, we will see the following:

```

forumbd=> select * from O_tags;
pk | tag | parent
----+---+-----
  5 | OpenBSD |
(1 row)

```

The record is present in both tables. A question worth asking is whether the rules are executed before the event or after the event. For example, is the newly created rule executed before INSERT or after INSERT? The answer is that rules in PostgreSQL are always executed before the event.

The INSTEAD OF option

Suppose now that we want to move all records with the field tag starting with the letter F or f in the F_tags table:

1. First, let's create a new table called F_tags:

```

forumbd=> create table F_tags (
pk integer not null primary key ,
tag text,
parent integer);
CREATE TABLE

```

2. Then let's create the new rule:

```

forumbd=> create or replace rule r_tags2
as on INSERT to tags

```